

 UNIVERSIDAD DON BOSCO	UNIVERSIDAD DON BOSCO FACULTAD DE INGENIERÍA ESCUELA DE COMPUTACIÓN		CICLO 02-2026
	MATERIA	DESARROLLO DE SOFTWARE PARA MOVILES	GUIA DE LABORATORIO # 4
	PRÁCTICA	PERMISOS DE UNA APP	

I. OBJETIVOS

- Abordar los diferentes tipos de permisos en una aplicación Android.
- Desarrollar una aplicación Android de ejemplo, para verificar el estado de un permiso determinado en Android.
- Desarrollar una aplicación Android de ejemplo, para gestionar un permiso determinado en tiempo de ejecución

II. INTRODUCCIÓN TEORICA

Permisos en Android

Las aplicaciones Android deben solicitar permiso para acceder a datos del usuario confidenciales, como contactos y SMS, o a determinadas funciones del sistema, como la cámara y el acceso a Internet. Cada permiso se identifica con una etiqueta única. Por ejemplo, en el manifiesto de una app que necesite enviar mensajes SMS debe incluirse esta línea:

```
<manifest ...>
  <uses-permission android:name="android.permission.SEND_SMS"/>
  ...
</manifest>
```

A partir de Android 6.0 (nivel de API 23), el usuario puede aprobar o rechazar algunos permisos durante el tiempo de ejecución. No obstante, sin importar qué versión de Android admita la aplicación, se debe declarar todas las solicitudes de permisos con un elemento **<uses-permission>** en el manifiesto. Si se otorga el permiso, la aplicación puede usar las funciones protegidas, de lo contrario, los intentos de acceder a esas funciones fallarán.

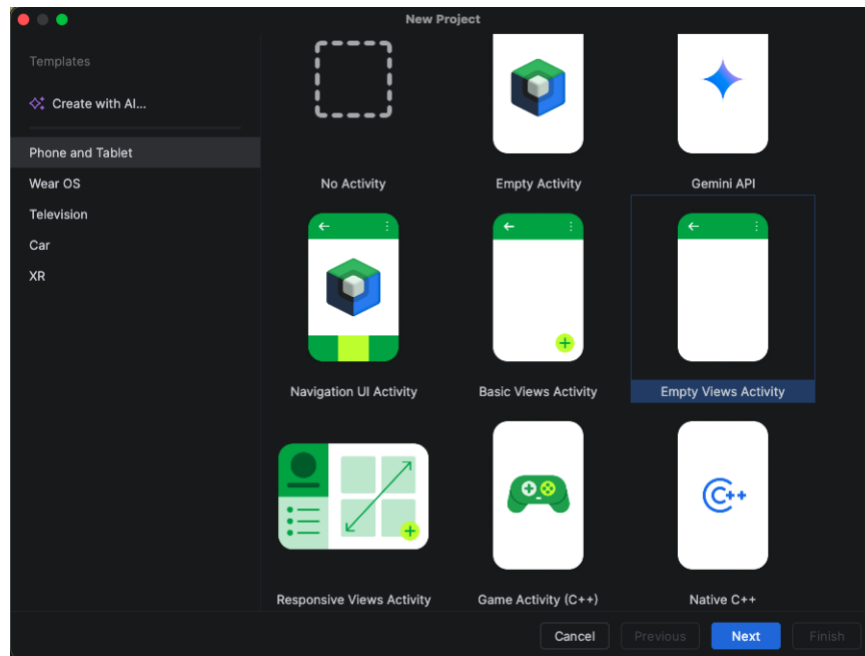
La aplicación también puede proteger sus propios componentes con permisos. Puede usar cualquiera de los permisos definidos por Android, tal como se indica en **android.Manifest.permission**, o un permiso que esté declarado en otra app.

Nota: Un permiso nuevo se declara con el elemento **<uses-permission>**

III. DESARROLLO DE LA PRÁCTICA

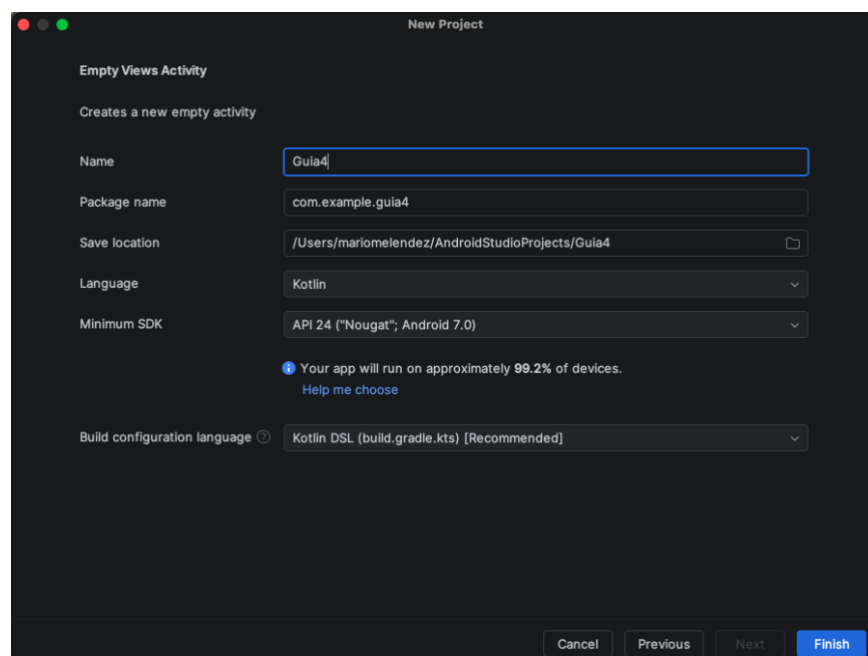
Parte 1 - Verificación de permisos

1. Cree un nuevo proyecto en Android Studio, asegúrese de seleccionar la plantilla **Empty Views Activity**.



2. Configure el proyecto con las siguientes características:

- **Nombre:** Guia4.
- **Ubicación:** por defecto (o use una ubicación de su preferencia).
- **Lenguaje:** Kotlin.
- **Minimun SDK:** API 24 Nougat.



3. Una vez finalizada la sincronización del proyecto, modifique el archivo **AndroidManifest.xml** para agregar un permiso para solicitar grabar audio y la cámara.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <!-- PERMISOS -->
    <uses-permission android:name="android.permission.RECORD_AUDIO" />
    <!-- PERMISOS -->

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.Guia4">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:windowSoftInputMode="adjustResize">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

- Declarar un permisos en el manifiesto ayuda a que la tienda de aplicaciones y usuarios sepan cual es el conjunto de permisos que puede solicitar una aplicación.
- Existen permisos que están estrictamente relacionados al hardware del dispositivo, algunos de ellos pueden consultarse en el siguiente enlace <https://developer.android.com/guide/topics/manifest/uses-feature-element?hl=es-419#permissions-features>.

4. Modifique el archivo **activity_main.xml** con el siguiente diseño.

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:id="@+id/main"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="24sp"
        android:textStyle="bold"
        android:textAlignment="center"
        android:text="Guía 04\n\nVerificación de permisos" />

</LinearLayout>

```

5. Por último, modifique el archivo **MainActivity.kt** con el siguiente código.

```

package com.example.guia4

import android.Manifest
import android.content.pm.PackageManager
import android.os.Bundle
import android.util.Log
import androidx.appcompat.app.AppCompatActivity
import androidx.core.content.ContextCompat

class MainActivity : AppCompatActivity() {

    private val TAG = "Guia4-Permisos"

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        // Comprobación de permisos
        comprobarEstadoPermiso()
    }
}

```

```

}

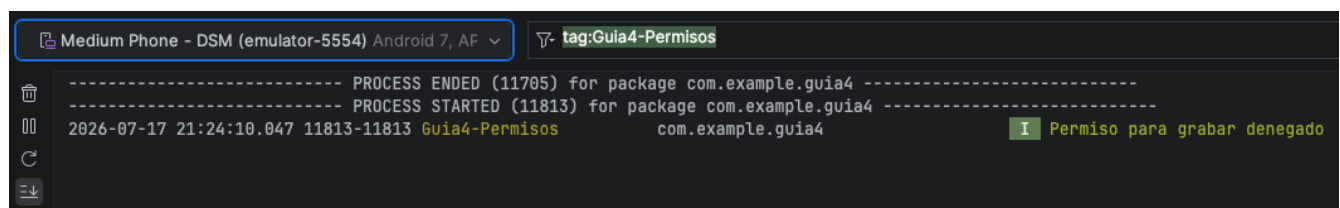
// 1. Comprobar estado permiso
private fun comprobarEstadoPermiso() {
    val estadoPermiso = ContextCompat.checkSelfPermission(
        this,
        Manifest.permission.RECORD_AUDIO
    )
    if (estadoPermiso != PackageManager.PERMISSION_GRANTED) {
        Log.i(TAG, "Permiso para grabar denegado")
    }
}

// 2. Configurar permisos
// 3. Solicitud de permiso
// 4. Manejo de respuesta de solicitud de permiso
}

```

- **ContextCompat** es una clase de la biblioteca AndroidX que proporciona métodos compatibles con distintas versiones de Android. Garantiza que el código funcione correctamente tanto en versiones antiguas como en las más recientes.
- La función **ContextCompat.checkSelfPermission()** comprueba si la aplicación tiene el permiso correspondiente, en este caso, permiso sobre el micrófono. Este método retorna un valor entero el cual indica si el permiso esta concedido (PERMISSION_GRANTED) o denegado (PERMISSION_DENIED).
 - **this** indica que el **Context** es la Actividad.
 - **Manifest.permission.RECORD_AUDIO** indica el permiso que se quiere comprobar, si se necesita comprobar mas de un permiso cada uno debe ser comprobado individualmente.
- Los permisos catalogados como peligrosos estan denegados por defecto hasta que son concedidos por el usuario en tiempo de ejecucion.

6. Ejecute la aplicación en el emulador o dispositivo móvil y compruebe que en el log del sistema que el que permiso para grabar audio esta denegado.



```

Medium Phone - DSM (emulator-5554) Android 7, API 24
tag:Guia4-Permisos

----- PROCESS ENDED (11705) for package com.example.guia4 -----
----- PROCESS STARTED (11813) for package com.example.guia4 -----
2026-07-17 21:24:10.047 11813-11813 Guia4-Permisos com.example.guia4 I Permiso para grabar denegado

```

Parte 2 – Gestion de permisos onRequestPermissionsResult()

1. Agregue una nueva variable al inicio de la clase **MainActivity.kt** de la siguiente manera.

```
class MainActivity : AppCompatActivity() {  
  
    private val TAG = "Guia4-Permisos"  
    private val CODIGO_SOLICITUD_GRABAR = 101  
  
    /*  
    *  
    */  
}
```

- Este ID servira para identificar la solicitud de permisos, puede tomar cualquier numero entero positivo.

2. Modifique el archivo **res/values/string.xml** y agregue las siguientes cadenas de texto.

```
<resources>  
    <string name="app_name">Guia4</string>  
    <string name="permiso_audio_denegado">Permiso para grabar denegado</string>  
    <string name="permiso_audio_concedido">Permiso para grabar concedido</string>  
    <string name="permiso_audio_denegado_usuario">Permiso para grabar denegado por el usuario</string>  
    <string name="permiso_audio_concedido_usuario">Permiso para grabar concedido por el usuario</string>  
    <string name="permiso_audio_requerido">Permiso para acceder al micrófono es requerido por esta App para grabar audio.</string>  
</resources>
```

3. Modifique el código de la clase **MainActivity.kt** en la seccion “**2. Configurar permisos**” de la siguiente manera.

```
// 2. Configurar permisos  
private fun configurarPermiso() {  
    val estadoPermiso = ContextCompat.checkSelfPermission(  
        this,  
        android.Manifest.permission.RECORD_AUDIO  
    )  
  
    if (estadoPermiso != PackageManager.PERMISSION_GRANTED) {  
        Log.i(TAG, getString(R.string.permiso_audio_denegado))  
  
        val mostrarRequest = ActivityCompat.shouldShowRequestPermissionRationale(  
            this,  
            android.Manifest.permission.RECORD_AUDIO  
        )  
    }  
}
```

```

);

if (mostrarRequest) {
    val builder = AlertDialog.Builder(this)
    builder.setMessage(getString(R.string.permiso_audio_requerido))
        .setTitle("Permiso requerido")

    builder.setPositiveButton("OK") { _, _ ->
        Log.i(TAG, "Seleccionado")
        solicitudPermiso()
    }
    builder.setNegativeButton("Cancelar", null)
    val dialog = builder.create()
    dialog.show()

} else {
    solicitudPermiso()
}
}
}

```

- El objetivo del metodo **configurarPermiso()** es comprobar si el permiso **RECORD_AUDIO** está concedido o no, si no lo está, pregunta al usuario si otorgarlo o denegarlo.
- La función **ActivityCompat.shouldShowRequestPermissionRationale()** retorna un valor de tipo **Boolean** según los casos:
 - **True** si el usuario ya rechazó el permiso anteriormente pero **NO** seleccionó la opcion **No volver a preguntar**.
 - **False** si es la primera vez que se va a solicitar el permiso o el usuario **SI** marcó la opcion **No volver a preguntar**.
- Un **AlertDialog** es un cuadro de dialogo el cual muestra opciones al usuario, para este ejemplo, se utiliza para mostrar al usuario un mensaje del porque es necesario conceder o denegar el permiso **RECORD_AUDIO**.

4. Agregue el siguiente codigo a la seccion **"3. Solicitud de permiso"** de la clase **MainActivity.kt**.

```

// 3. Solicitud de permiso
private fun solicitudPermiso() {
    ActivityCompat.requestPermissions(
        this,
        arrayOf(android.Manifest.permission.RECORD_AUDIO),

```

```

        CODIGO_SOLICITUD_GRABAR
    )
}

```

- La función **ActivityCompat.requestPermissions()** muestra un cuadro de dialogo para solicitar el permiso al usuario. El parámetro **CODIGO_SOLICITUD_GRABAR** identifica de manera única la solicitud del permiso.

5. Agregue el siguiente código a la sección “4. Manejo de respuesta de solicitud de permiso” de la clase **MainActivity.kt**.

```

// 4. Manejo de respuesta de solicitud de permiso
override fun onRequestPermissionsResult(
    requestCode: Int,
    permissions: Array<String>,
    grantResults: IntArray
) {
    super.onRequestPermissionsResult(requestCode, permissions, grantResults)

    when (requestCode) {
        CODIGO_SOLICITUD_GRABAR -> {
            if (grantResults.isEmpty() || grantResults[0] != PackageManager.PERMISSION_GRANTED) {

                Log.i(TAG, getString(R.string.permiso_audio_denegado_usuario))

                Toast.makeText(
                    applicationContext,
                    getString(R.string.permiso_audio_denegado_usuario),
                    Toast.LENGTH_SHORT
                ).show()

                /* Código cuando el usuario deniegue el permiso */
            } else {
                Log.i(TAG, getString(R.string.permiso_audio_concedido_usuario))

                Toast.makeText(
                    applicationContext,
                    getString(R.string.permiso_audio_concedido_usuario),
                    Toast.LENGTH_SHORT
                ).show()

                /* Código cuando el usuario conceda el permiso */
            }
        }
    }
}

```

```

    }
}
}
}

```

- El callback **onRequestPermissionsResult()** se ejecuta cuando el usuario responde a la solicitud del permiso, los parámetros corresponden al identificador de la solicitud del permiso, los permisos que se están solicitando y el resultado de cada permiso.

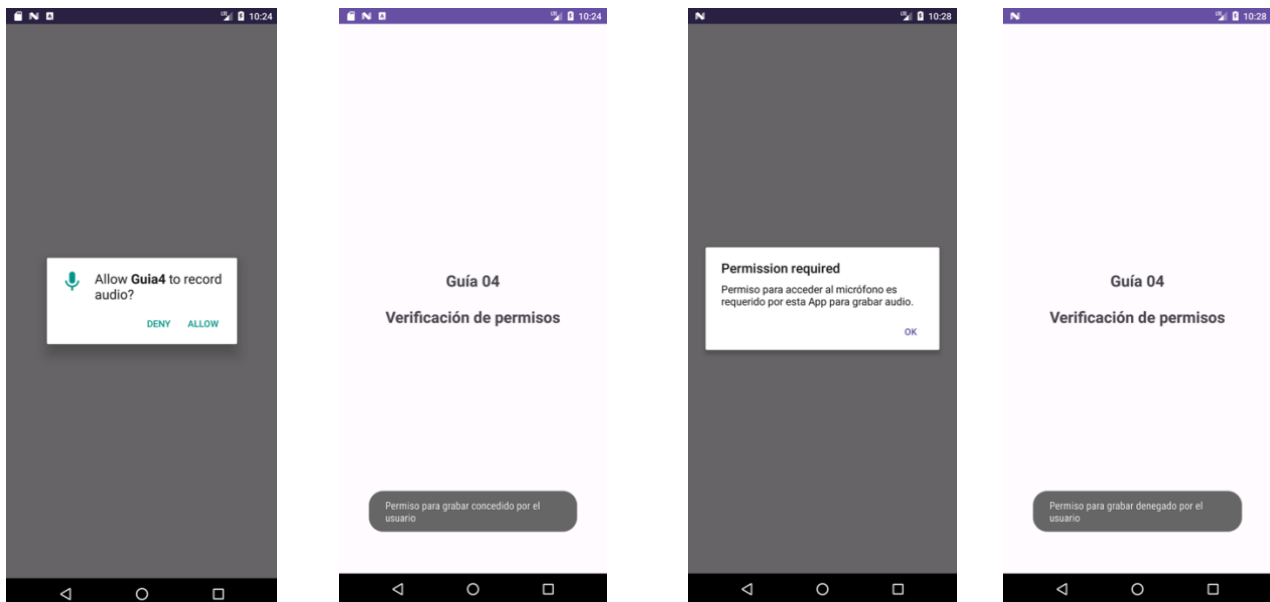
6. Modifique el callback **onCreate()** de la clase **MainActivity.kt** para incluir la llamada a la función **configurarPermiso()**.

```

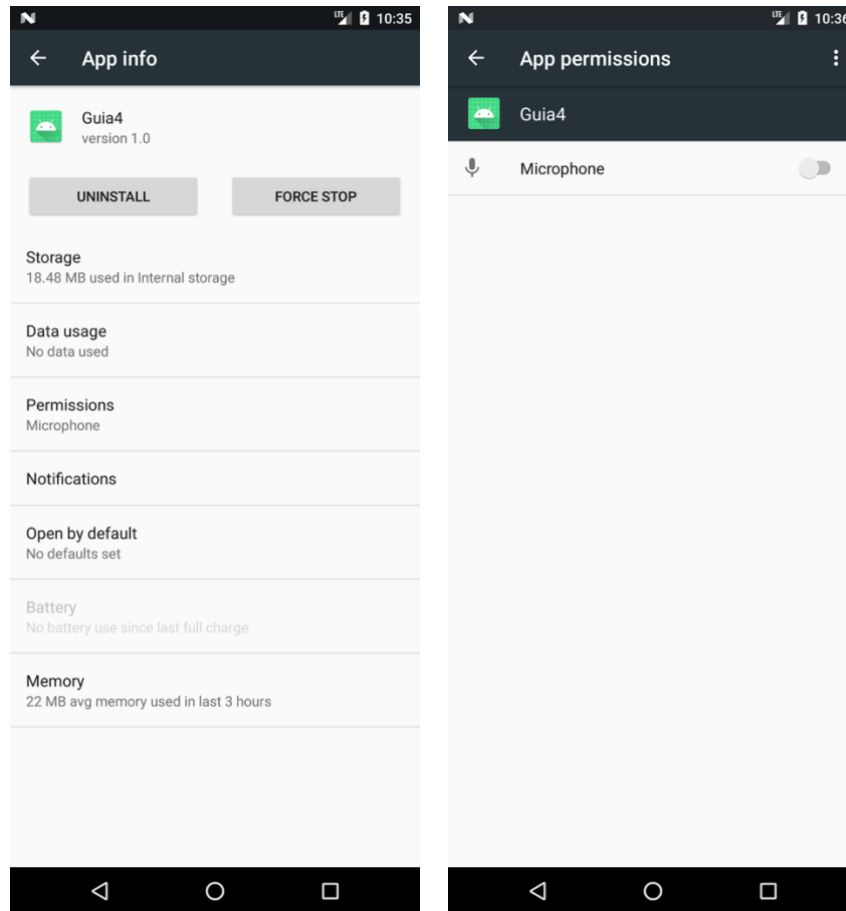
override fun onCreate (savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
    // Comprobación de permisos
    configurarPermiso()
}

```

6. Ejecute la aplicación y compruebe que en tiempo de ejecución la aplicación le solicite permisos para grabar archivos de audio.



7. Si otorgo el permiso, es posible revocar el permiso desde la configuración de la aplicación. Para ello, en la configuración de Android, ingrese a las aplicaciones instaladas y ubique la app de la **Guía4**. Posteriormente, en la sección de permisos revoque el permiso otorgado.



8. Verifique el log del sistema para visualizar los mensajes escritos por la aplicación con el tag **Guia04 – Permisos**.

```

Medium Phone - DSM (emulator-5554) Android 7, AF  tag:Guia4-Permisos
----- PROCESS STARTED (15341) for package com.example.guia4 -----
2026-07-17 22:37:56.103 15341-15341 Guia4-Permisos com.example.guia4 I Permisos para grabar denegado
2026-07-17 22:37:57.839 15341-15341 Guia4-Permisos com.example.guia4 I Permisos para grabar denegado por el usuario
----- PROCESS ENDED (15341) for package com.example.guia4 -----
----- PROCESS STARTED (15451) for package com.example.guia4 -----
2026-07-17 22:38:08.280 15451-15451 Guia4-Permisos com.example.guia4 I Permisos para grabar denegado
2026-07-17 22:38:11.105 15451-15451 Guia4-Permisos com.example.guia4 I Selecccionado
2026-07-17 22:38:12.272 15451-15451 Guia4-Permisos com.example.guia4 I Permisos para grabar concedido por el usuario

```

Parte 3 – Gestion de permisos API Activity Result

1. Agregue el siguiente permiso en el archivo **AndroidManifest.xml**.

```

<!-- PERMISOS -->
<uses-permission android:name="android.permission.RECORD_AUDIO" />

<uses-permission android:name="android.permission.CAMERA" />

<uses-feature

```

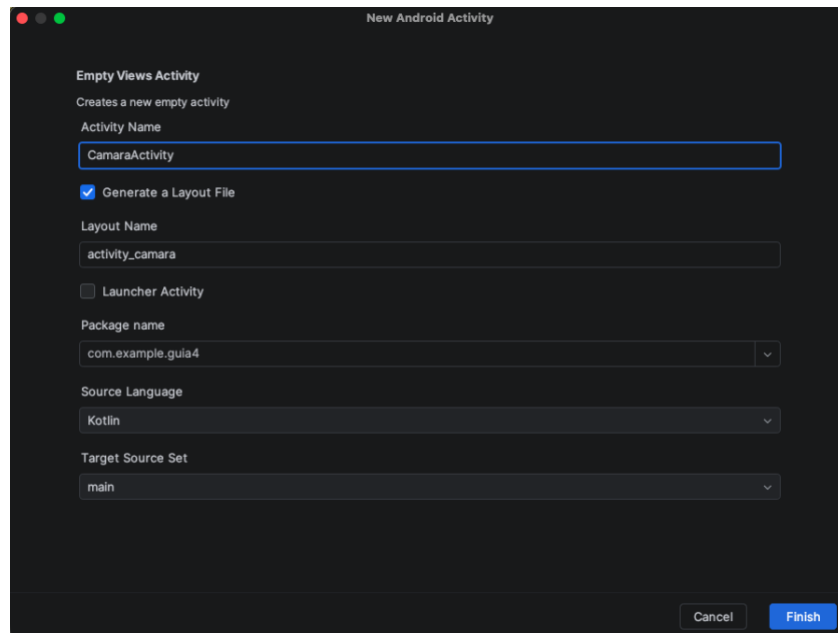
```

    android:name="android.hardware.camera"
    android:required="false" />
<!-- PERMISOS -->

```

- El permiso **CAMERA** esta asociado con el hardware, por lo que es necesario agregar el tag **uses-feature**.
- La propiedad **required** indica que a pesar de que se solicita utilizar la camera no sera requerida para los dispositivos que no tengan el hardware.

2. Agregue una nueva **Empty Views Activity** con el nombre **CamaraActivity**.



3. Modifique el diseño del archivo **activity_camara.xml** de la siguiente manera.

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:id="@+id/main"
    tools:context=".CamaraActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="24sp"

```

```

        android:textStyle = "bold"
        android:textAlignment = "center"
        android:text = "Verificación de permisos\n\nCamara" />

```

```
</LinearLayout>
```

5. Modifique el archivo **res/values/string.xml** y agregue las siguientes cadenas de texto.

```

<string name="permiso_camara_denegado">Permiso para camara denegado</string>
<string name="permiso_camara_concedido">Permiso para camara concedido</string>
<string name="permiso_camara_requerido">Permiso para acceder a la camara es requerido por esta App para
tomar fotos.</string>

```

6. Modifique el código de la clase **CamaraActivity.kt** con el siguiente código.

```

package com.example.guia4

import android.Manifest
import android.content.pm.PackageManager
import android.os.Bundle
import android.util.Log
import androidx.activity.result.contract.ActivityResultContracts
import androidx.appcompat.app.AppCompatActivity
import androidx.core.app.ActivityCompat
import androidx.core.content.ContextCompat

class CamaraActivity : AppCompatActivity() {

    private val TAG = "Guia4-Permisos"

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_camara)
        comprobarPermiso()
    }

    private val solicitarPermiso = registerForActivityResult(
        ActivityResultContracts.RequestPermission()
    ) { concedido ->

        if (concedido) {
            Log.i(TAG, getString(R.string.permiso_camara_concedido))
        }
    }
}

```

```

        Log.i(TAG, "Abriendo camara...")
    } else {
        Log.i(TAG, getString(R.string.permiso_camara_denegado))
    }
}

private fun comprobarPermiso() {

    when {

        ContextCompat.checkSelfPermission(
            this,
            Manifest.permission.CAMERA
        ) == PackageManager.PERMISSION_GRANTED -> {
            Log.i(TAG, "Abriendo camara...")
        }

        ActivityCompat.shouldShowRequestPermissionRationale(
            this,
            Manifest.permission.CAMERA
        ) -> {

            mostrarDialogo()
        }

        else -> {

            solicitarPermiso.launch(
                Manifest.permission.CAMERA
            )
        }
    }
}

private fun mostrarDialogo() {

    val builder = androidx.appcompat.app.AlertDialog.Builder(this)

    builder.setMessage(getString(R.string.permiso_camara_requerido))
        .setTitle("Permission required")
    builder.setPositiveButton("OK") { _, _ ->

```

```

        Log.i(TAG, "Seleccionado")
        solicitarPermiso.launch(
            Manifest.permission.CAMERA
        )
    }
    builder.setNegativeButton("Cancelar", null)
    val dialog = builder.create()
    dialog.show()
}
}

```

- **registerForActivityResult()** registra una acción que espera un resultado, para este ejemplo, si el usuario concede o deniega el permiso.
- El metodo **comprobarPermiso()** gestiona la siguiente:
 - Comprobar el estado del permiso con **checkSelfPermission()**.
 - Mostrar explicacion al usuario sobre el permiso con **shouldShowRequestPermissionRationale()**.
 - Lanzar la solicitud del permiso.

7. Modifique el archivo **activity_main.xml** para agregar un nuevo boton.

```

<Button
    android:id="@+id/btnCamara"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="20dp"
    android:text="Abrir Activity Camara" />

```

8. Modifique el la clase **MainActivity.kt** para agregar la referencia al boton **btnCamara** y su respectivo listener.

```

lateinit var btnCamera: Button

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    // Comprobación de permisos
    configurarPermiso()

    btnCamera = findViewById(R.id.btnCamara)
}

```

```

btnCamera.setOnClickListener {
    val intent = Intent(this, CamaraActivity::class.java)
    startActivity(intent)
}
}

```

9. Compruebe el funcionamiento de la aplicación, revise en el log del sistema los mensajes de cada parte del flujo de interacción de la solicitud de permisos.

IV. DISCUSIÓN DE RESULTADOS

Se le solicita que simule una aplicación grabadora de audio, para ello agregue tres controles:

- Un **TextView** con el texto **Grabadora de Audio**.
- Un **Button** con el texto **Iniciar Grabación**.
- Un **TextView** que muestre el estado actual **Esperando permiso, Grabando, Permiso denegado**.

Codifique la aplicación de tal forma que cumpla las siguientes condiciones:

1. Al iniciar la aplicación compruebe el estado del permiso **RECORD_AUDIO**, si este esta concedido muestre el botón **Iniciar Grabación**, por el contrario, solicite al usuario el permiso correspondiente.
2. Cuando el usuario presione el botón **Iniciar Grabación**:
 - a. Si el permiso **esta concedido** cambie el **TextView** correspondiente al estado de la grabación a **Grabando**.
 - b. Si el permiso **no esta concedido**, realice la solicitud al usuario para su autorización o denegación.
3. Si el usuario deniega el permiso, oculte o bloquee el botón **Iniciar Grabación**.

NOTA: para deshabilitar un botón utilice la propiedad **isEnabled**, ejemplo:

```

btnIniciarGrabacion.isEnabled = false

```

V. BIBLOGRAFÍA

- Documentación Oficial Android - Cómo declarar permisos de la app.
<https://developer.android.com/training/permissions/declaring7hkes-419>
- Documentación Oficial Android - Solicitud permisos de tiempo de ejecución.
<https://developer.android.com/training/permissions/requesting7hkes-419>
- Manifest Permission.
<https://developer.android.com/reference/android/Manifest.permission>
- Permission
<https://developer.android.com/reference/kotlin/android/Manifest.permission>